

Data and Artificial Intelligence

Cyber Shujaa Program

Week 9 Assignment

MLOPs

Student Name: Violet Joy

Student ID: cs-da01-25025

Table of Content

Data and Artificial Intelligence	1
Cyber Shujaa Program.....	1
Week 9 Assignment MLOPs	1
Abbreviations	3
Introduction	4
Tasks Completed	5
2.1: Loading and Exploring the Dataset	5
2.2: Apply Pre-processing.....	8
2.3: Split the Data into Training and Test Sets (80/20 Split)	10
2.4: Use a KNeighborsRegressor	11
2.5: Apply GridSearchCV to Tune Hyperparameters	13
2.6: Use 5-Fold Cross-Validation	14
2.7: Define an Evaluation Metric: R^2 Score	16
2.8: Save the Trained Pipeline Using <code>pickle</code> or <code>joblib</code>	17
Conclusion.....	19

Abbreviations

- MedInc: median income in block group
- HouseAge: median house age in block group
- AveRooms: average number of rooms per household
- AveBedrms: average number of bedrooms per household
- Population: block group population
- AveOccup: average number of household members
- Latitude: block group latitude
- Longitude: block group longitude

Introduction

In today's data-driven economy, accurately forecasting housing prices is a valuable capability for real estate firms, urban planners, policy makers, and financial institutions. Predictive modeling can uncover hidden patterns in large-scale housing datasets, enabling more informed decisions in areas such as investment strategy, market analysis, mortgage lending, and public housing development.

This project focuses on building a complete machine learning pipeline to predict median house values using the **California Housing dataset**, a well-known benchmark dataset derived from the 1990 U.S. Census. The dataset is rich in demographic, economic, and geographic information about California's block groups, making it ideal for training regression models.

This project not only demonstrates the technical process of building and evaluating a regression model but also highlights the business potential of predictive analytics in the housing sector. By automating price prediction, stakeholders can make faster, data-informed decisions that minimize risk and maximize value.

Moreover, the emphasis on **clean architecture, reusability, and deployment** aligns with modern machine learning operations (MLOps) practices—crucial for transitioning from prototype to production in both academic research and business use cases.

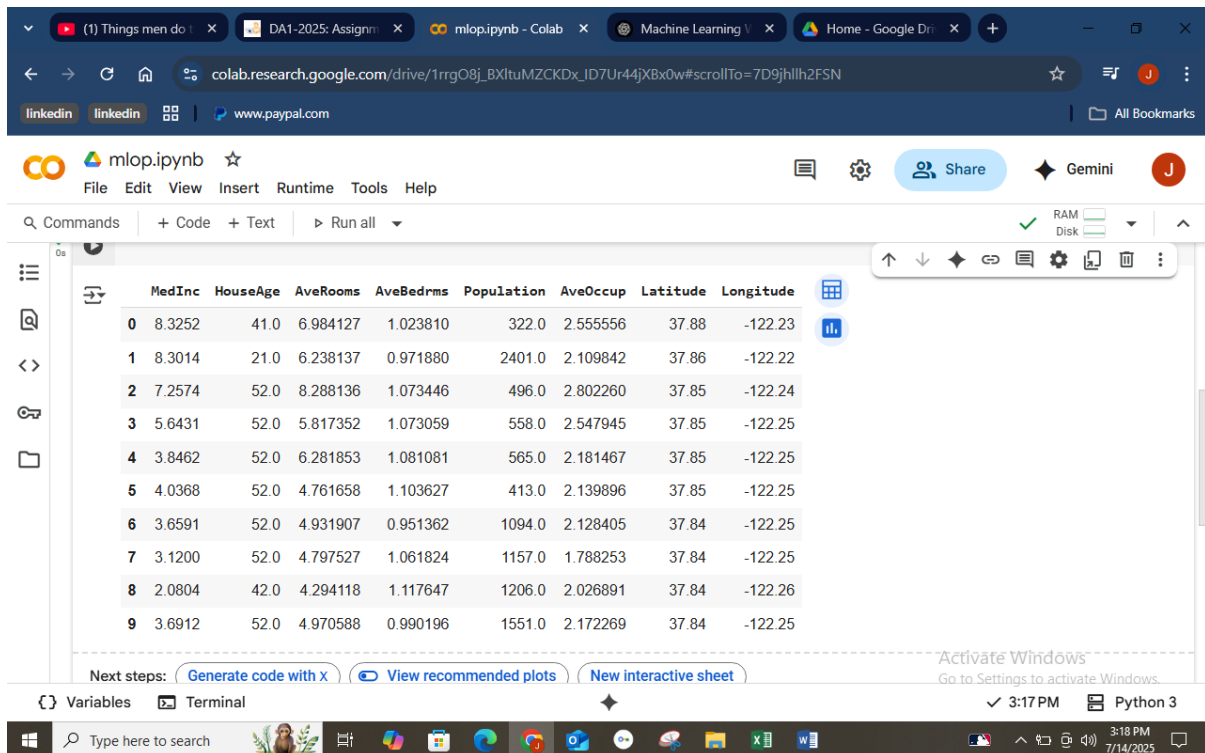
Tasks Completed

2.1: Loading and Exploring the Dataset

To begin the project, the California Housing dataset was loaded directly from Scikit-learn using the built-in `fetch_california_housing()` function. This utility fetches a clean and well-structured version of the dataset derived from the 1990 U.S. Census. It contains over 20,000 observations, each representing a block group in California, with a total of 8 numerical features and one continuous target variable: the median house value.

Once loaded into a Pandas DataFrame, basic exploratory data analysis (EDA) was conducted to understand the dataset structure and content. This included:

- **Inspecting the first few rows** using `X.head()` to verify data import.
- **Checking the shape** of the dataset with `X.shape` to understand its size.
- **Reviewing data types and null values** with `X.info()` and `X.isnull().sum()` to ensure data integrity.
- **Generating summary statistics** using `X.describe()` to analyze the distribution of values across features.



	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25
5	4.0368	52.0	4.761658	1.103627	413.0	2.139896	37.85	-122.25
6	3.6591	52.0	4.931907	0.951362	1094.0	2.128405	37.84	-122.25
7	3.1200	52.0	4.797527	1.061824	1157.0	1.788253	37.84	-122.25
8	2.0804	42.0	4.294118	1.117647	1206.0	2.026891	37.84	-122.26
9	3.6912	52.0	4.970588	0.990196	1551.0	2.172269	37.84	-122.25

Fig 2.1.1: Output of displaying the first ten rows of the dataset

Fig 2.1.1: Output of displaying the first ten rows of the dataset

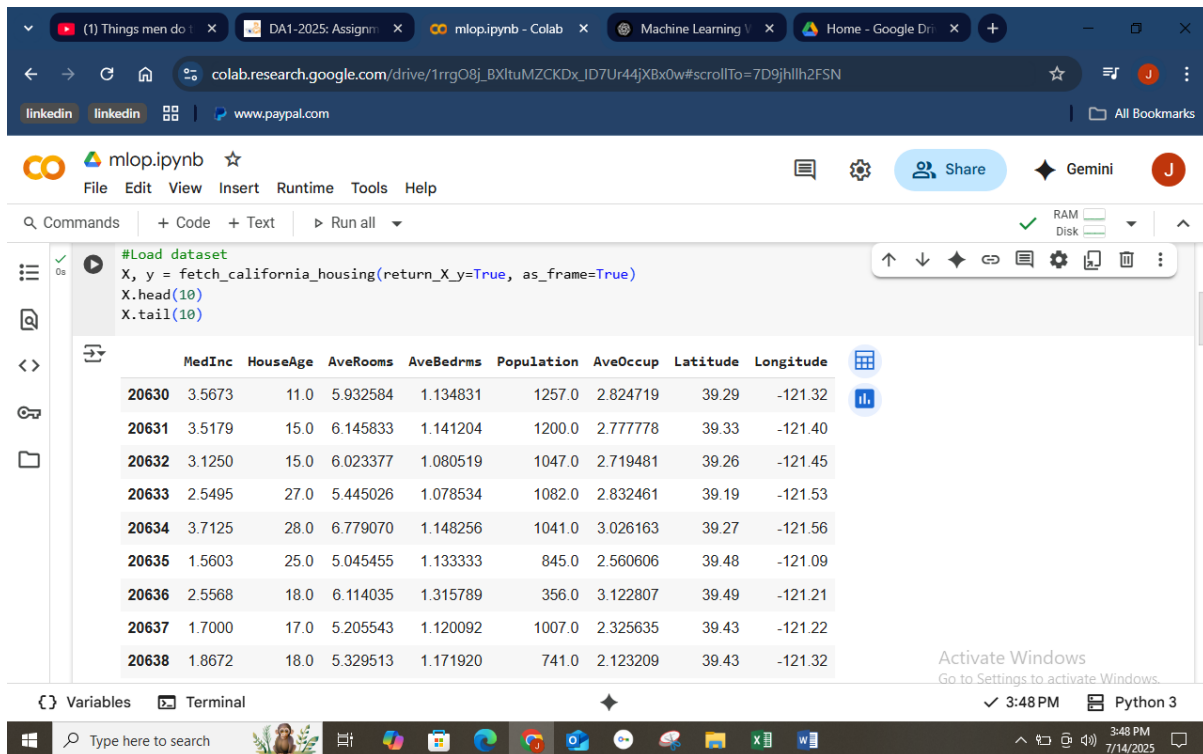


Fig 2.1.2: Output of displaying the last ten rows of the dataset

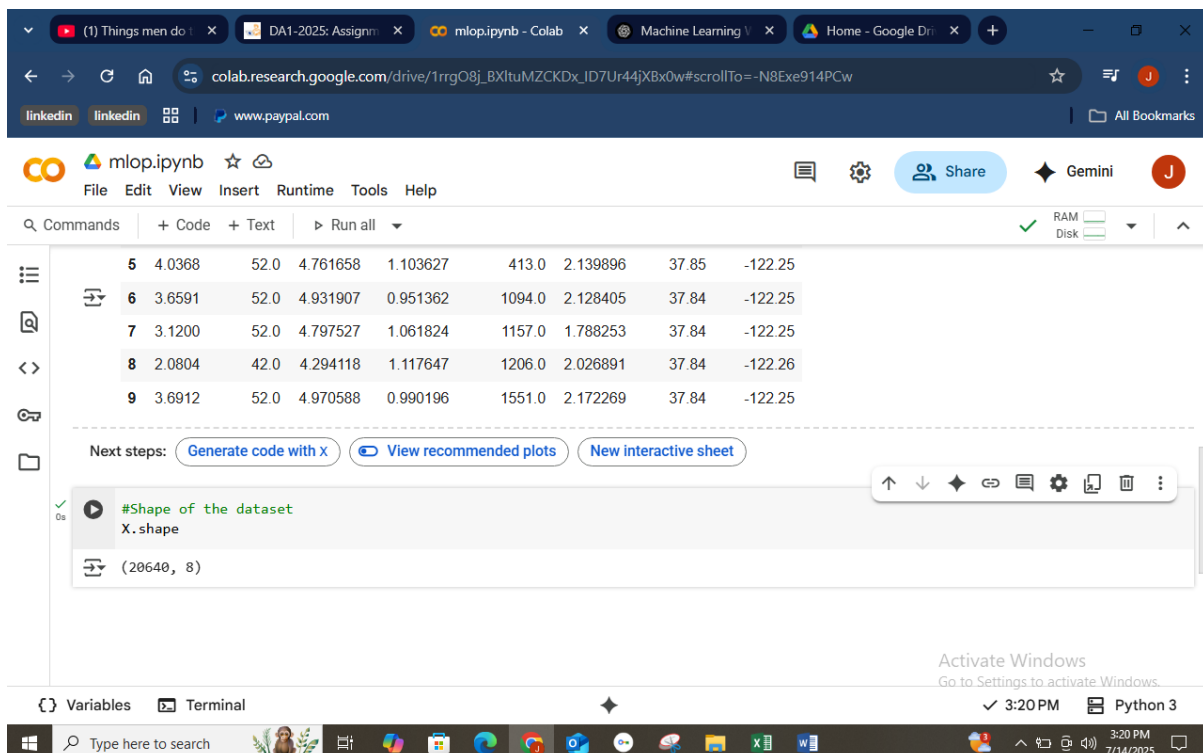


Fig 2.1.3: Output of shape of dataset

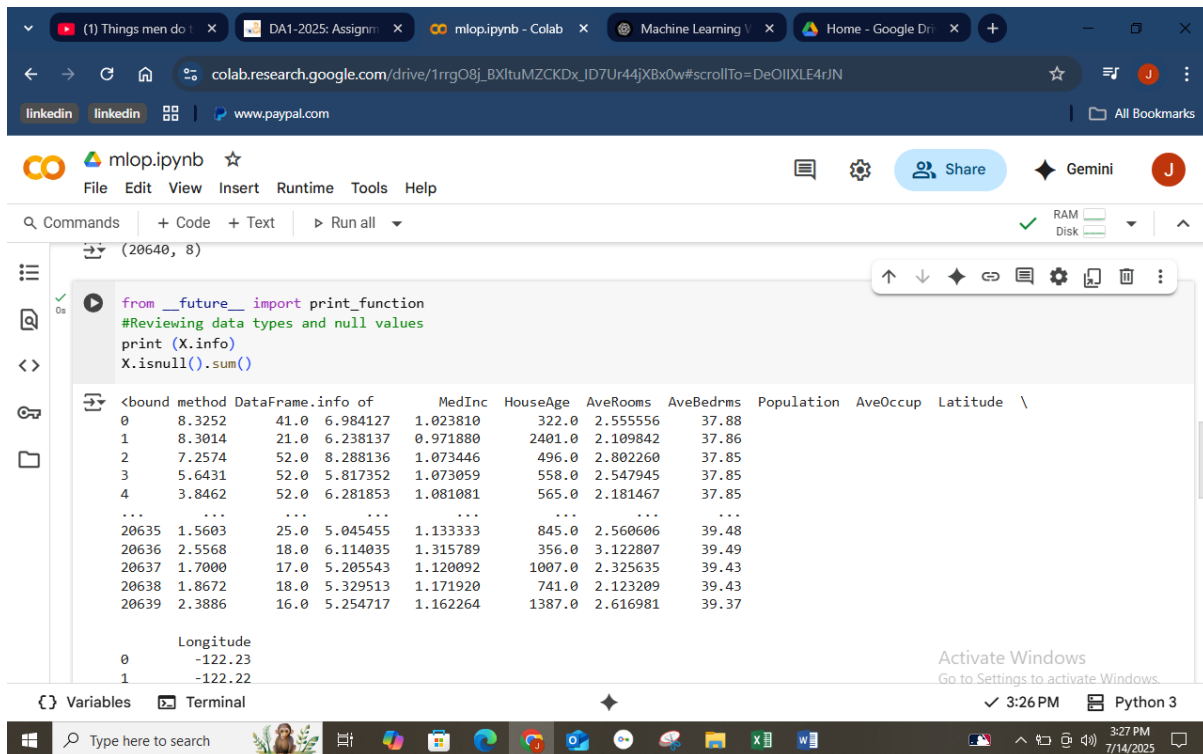


Fig 2.1.4:Output of reviewing dataset

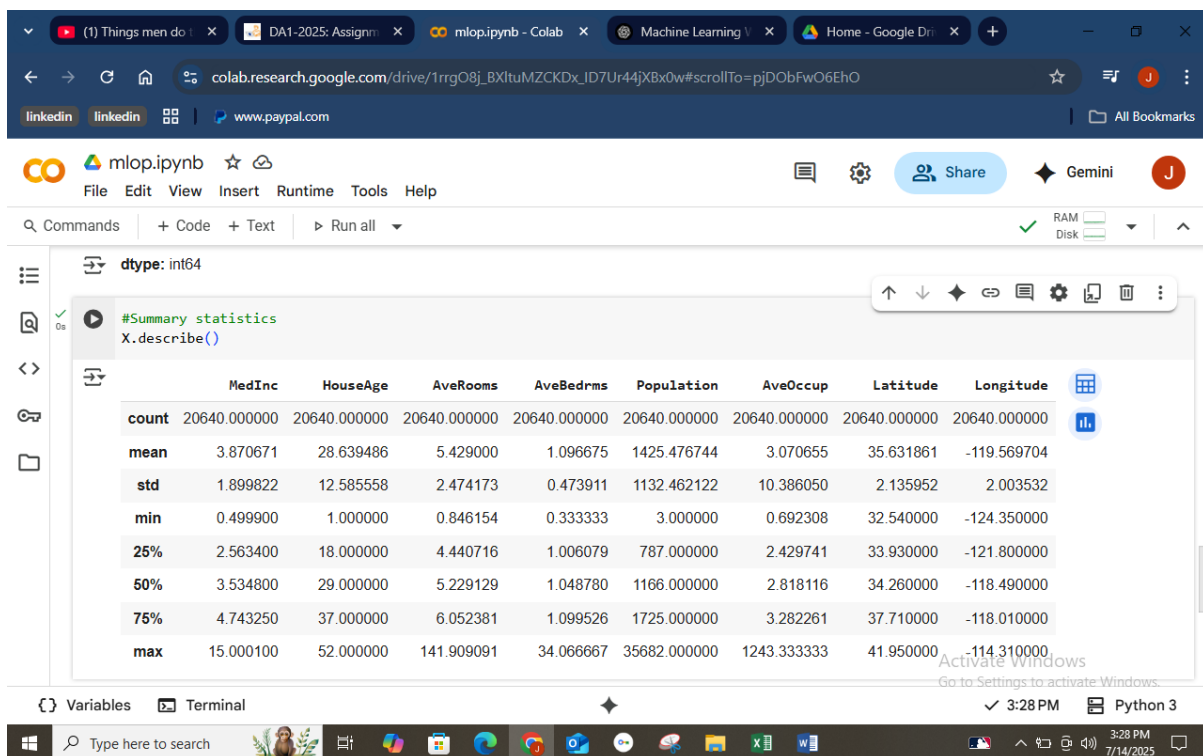


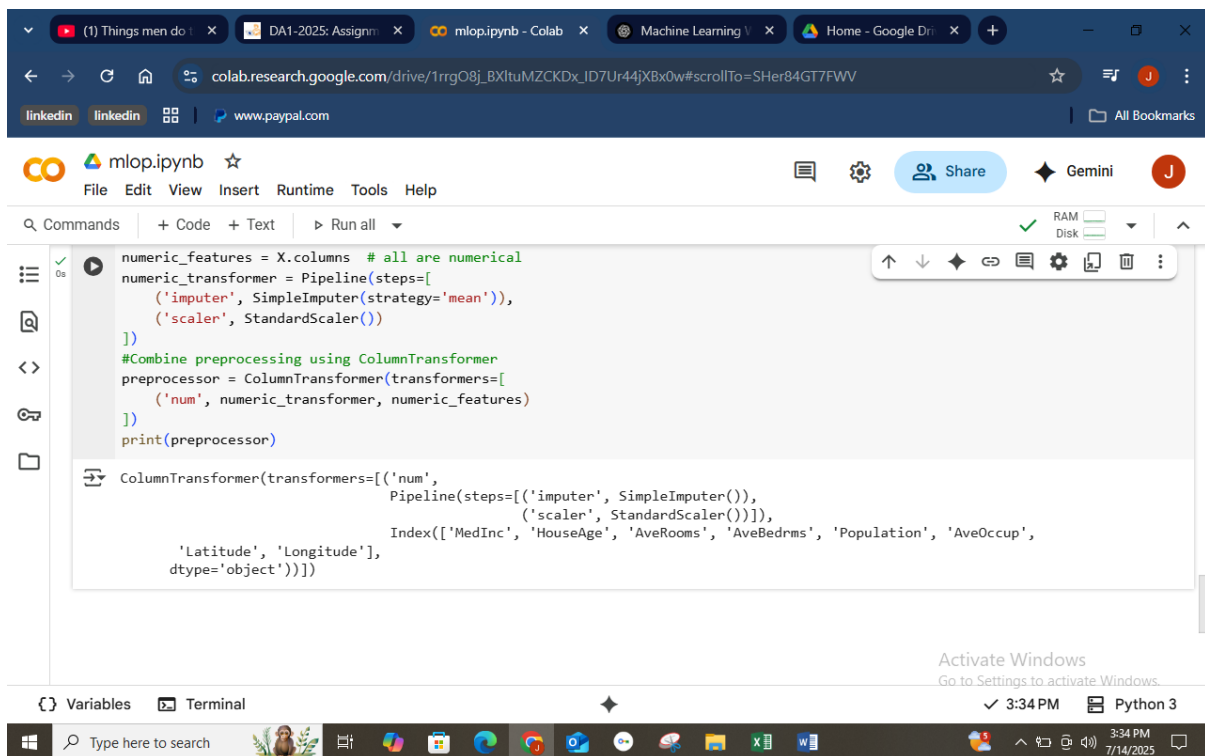
Fig 2.1.5:Output of summary statistics

2.2: Apply Pre-processing

Preprocessing is a crucial step in preparing the dataset for machine learning. It ensures that the data is cleaned, scaled, and transformed appropriately so that the model can learn effectively. In this project, Scikit-learn's ColumnTransformer was used to automate and streamline preprocessing for multiple feature columns in a unified pipeline.

Since the California Housing dataset consists entirely of **numerical features**, the main preprocessing step applied was **feature scaling**. This is particularly important for distance-based algorithms like K-Nearest Neighbors, where unscaled features could lead to biased predictions due to differences in feature magnitudes.

ColumnTransformer allows the application of different preprocessing techniques to specified subsets of features. Although all features in this dataset are numerical and receive the same transformation (scaling), using ColumnTransformer ensures scalability and flexibility, especially if future versions of the dataset contain mixed data types (e.g., categorical or textual data).

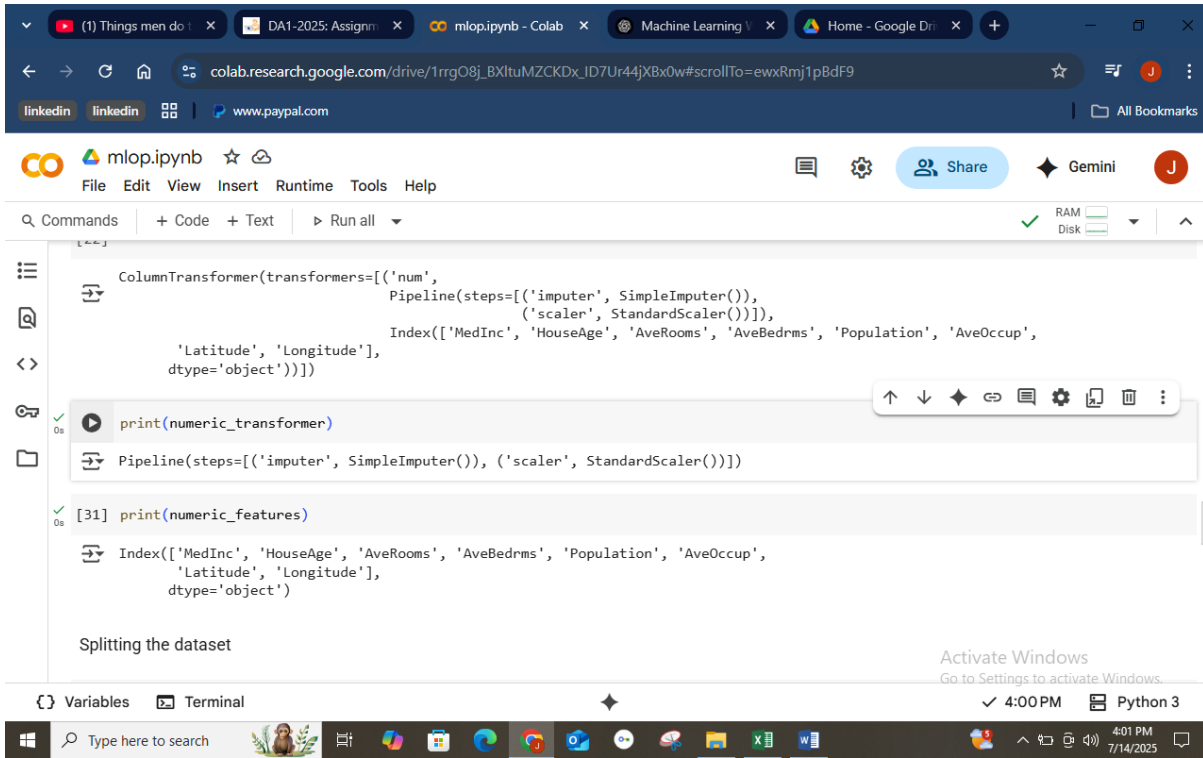


```

numeric_features = X.columns # all are numerical
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])
#Combine preprocessing using ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features)
])
print(preprocessor)

ColumnTransformer(transformers=[('num',
    Pipeline(steps=[('imputer', SimpleImputer()),
                    ('scaler', StandardScaler())]),
    Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
          'Latitude', 'Longitude'],
          dtype='object')))])
  
```

Fig 2.2.1:Output of column transformers



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the name 'mlop.ipynb', and various menu options like File, Edit, View, Insert, Runtime, Tools, and Help. The left sidebar shows a file explorer with a folder icon. The main area displays the following code and its output:

```
ColumnTransformer(transformers=[('num',
                                Pipeline(steps=[('imputer', SimpleImputer()),
                                                  ('scaler', StandardScaler())]),
                                Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
                                      'Latitude', 'Longitude'],
                                      dtype='object')))])

print(numeric_transformer)

Pipeline(steps=[('imputer', SimpleImputer()), ('scaler', StandardScaler())])

[31] print(numeric_features)

Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
      'Latitude', 'Longitude'],
      dtype='object')
```

Below the code, the text "Splitting the dataset" is visible. The bottom status bar shows "Variables", "Terminal", and "Python 3". The Windows taskbar at the very bottom displays the search bar, task view, and several application icons, along with the system clock showing 4:01 PM on 7/14/2025.

Fig 2.2.2: Output of `numeric_transformers` and `numeric_features`

2.3: Split the Data into Training and Test Sets (80/20 Split)

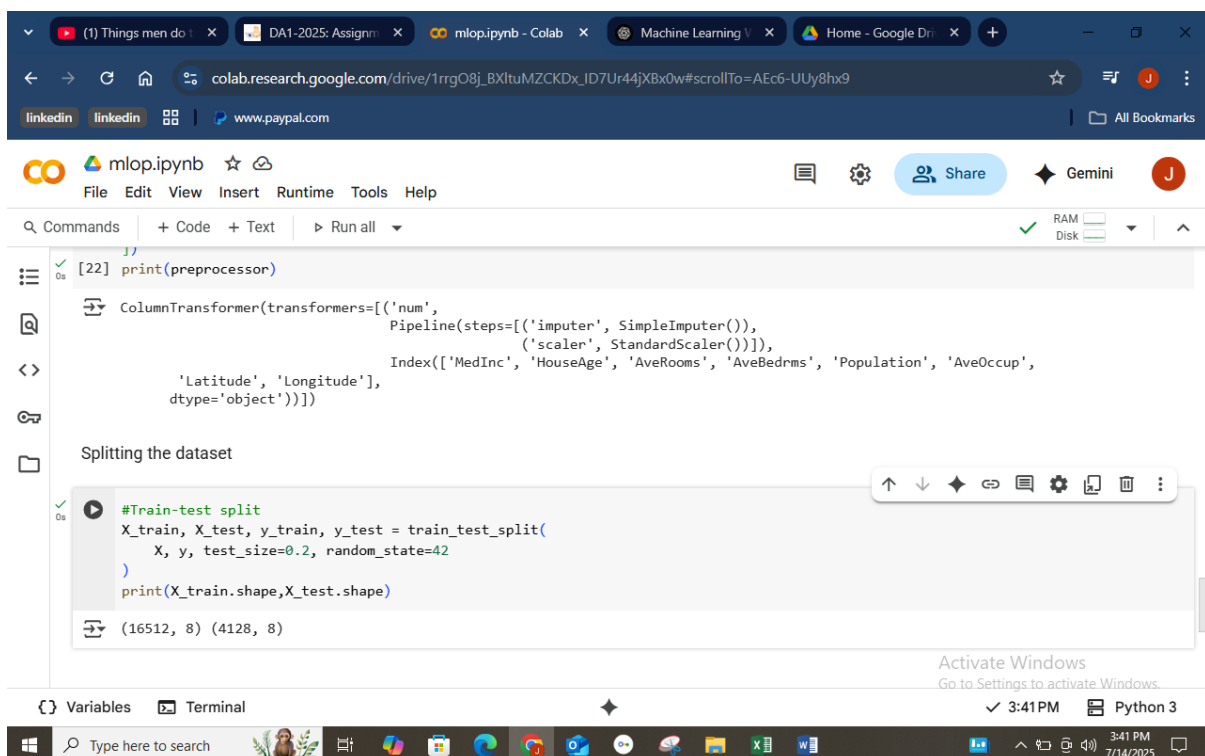
To evaluate how well the model generalizes to unseen data, the dataset was split into **training** and **test** subsets using an 80/20 ratio. This is a standard practice in machine learning that ensures the model is trained on the majority of the data while preserving a separate, untouched portion for performance evaluation.

The training set (80%) is used to fit and tune the model, including all preprocessing and cross-validation. The test set (20%) is withheld during training and is only used at the end of the workflow to provide an unbiased estimate of how the model will perform on real-world, unseen data.

Why the 80/20 Split?

- **80% Training Data:** Provides sufficient data for the model to learn underlying patterns.
- **20% Test Data:** Offers a reliable sample for evaluating model performance without leakage from training.

By carefully splitting the data before any model training, I prevent overfitting and ensure that our performance metrics reflect how well the model is likely to perform in practical applications.



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the file name 'mlop.ipynb', and various icons for sharing and settings. The notebook content is divided into two cells. The first cell contains a Python code snippet that defines a ColumnTransformer with a Pipeline of an imputer and a scaler, and an Index of feature names. The second cell, titled 'Splitting the dataset', contains a code snippet that uses train_test_split to split the data into training and test sets. The output of the second cell shows the shapes of the training and test sets: (16512, 8) for the training set and (4128, 8) for the test set.

```
[22] print(preprocessor)

ColumnTransformer(transformers=[('num',
                                Pipeline(steps=[('imputer', SimpleImputer()),
                                                  ('scaler', StandardScaler())]),
                                Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
                                       'Latitude', 'Longitude'],
                                       dtype='object')))])

Splitting the dataset

#Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(X_train.shape, X_test.shape)

(16512, 8) (4128, 8)
```

Fig 2.3.1: Output of splitting the dataset

2.4: Use a KNeighborsRegressor

For this project, the **K-Nearest Neighbors Regressor (KNeighborsRegressor)** was selected as the predictive model. This is a **non-parametric, instance-based learning algorithm** that makes predictions based on the average of the target values of the nearest neighbors in the feature space.

K-Nearest Neighbors (KNN) is especially suitable for datasets where the relationship between features and target is **non-linear** and **complex**, as it does not make any assumptions about the underlying data distribution.

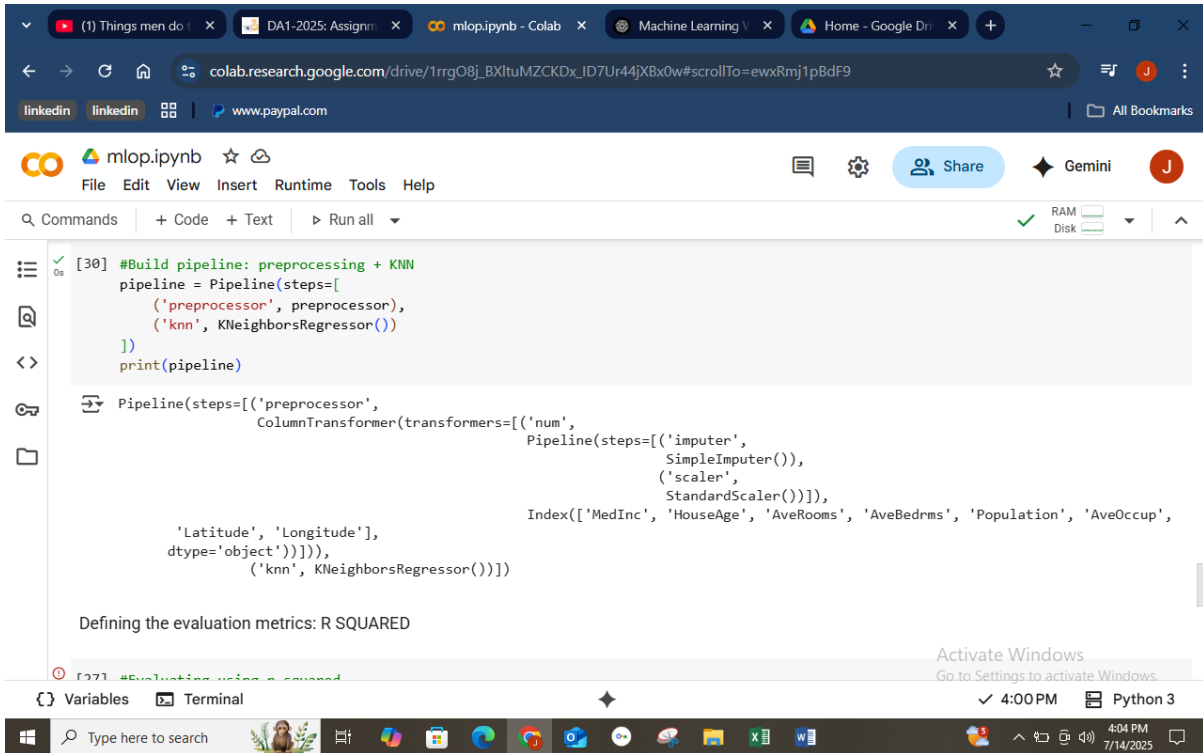
How It Works:

- For a given test point, the model identifies the **k closest training samples** (based on a chosen distance metric).
- The prediction is computed as the **average of the target values** of these neighbors.
- The closeness of samples is typically measured using either **Euclidean distance (p=2)** or **Manhattan distance (p=1)**.

Key Characteristics:

- **Simplicity:** Easy to understand and implement.
- **Flexibility:** Can adapt to complex data patterns if k is well-tuned.
- **No training phase:** Stores the training data and performs computation only during prediction (lazy learning).
- **Sensitive to feature scales:** Hence the importance of preprocessing with standardization.

The use of KNeighborsRegressor in this project demonstrates how intuitive, non-parametric models can be powerful tools for real estate value prediction when combined with proper preprocessing and tuning



colab.research.google.com/drive/1rrgO8j_BXltuMZCKDx_ID7Ur44jXBx0w#scrollTo=ewxRmj1pBdF9

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[30] #Build pipeline: preprocessing + KNN
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('knn', KNeighborsRegressor())
])
print(pipeline)
```

```
Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
                                                                    SimpleImputer()),
                                                                    ('scaler',
                                                                    StandardScaler())]),
                                                  Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
                                                                    'Latitude', 'Longitude'],
                                                                    dtype='object')))]),
                ('knn', KNeighborsRegressor())])
```

Defining the evaluation metrics: R SQUARED

Activate Windows
Go to Settings to activate Windows.

Variables Terminal 4:00 PM Python 3

Fig 2.4.1: Output of build pipeline

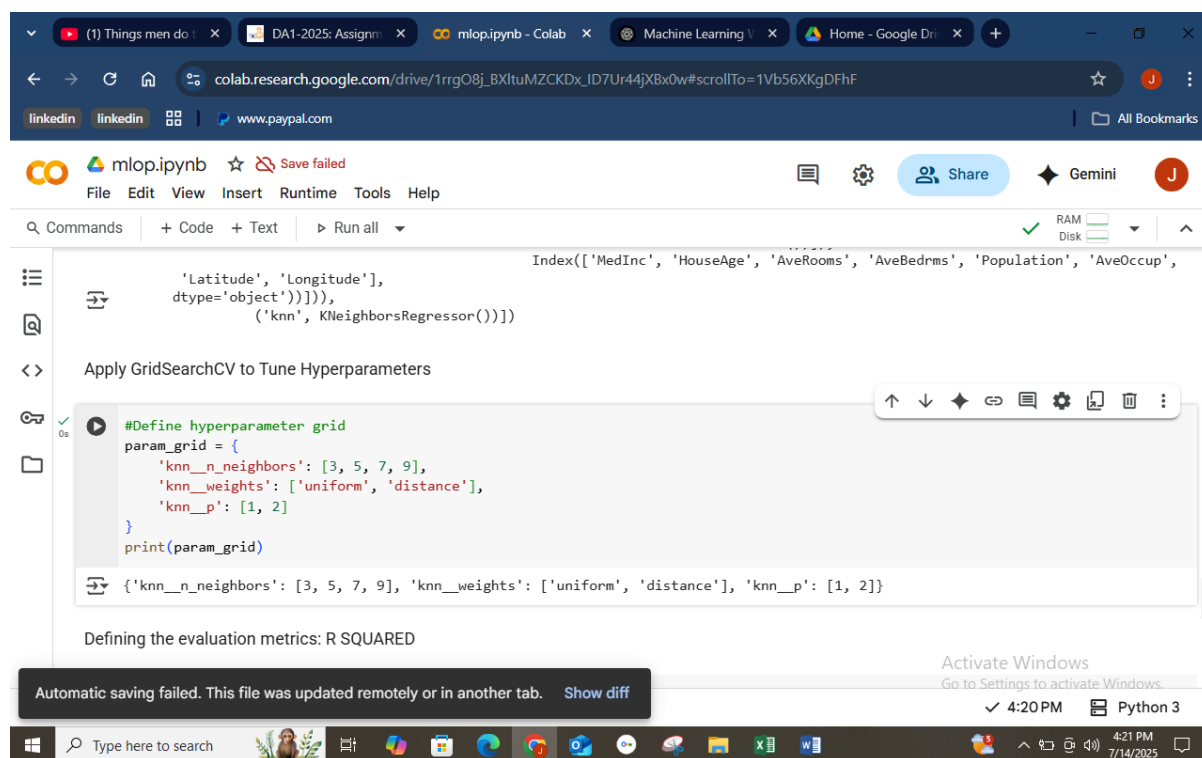
2.5: Apply GridSearchCV to Tune Hyperparameters

To optimize the performance of the **KNeighborsRegressor**, a comprehensive **hyperparameter tuning** process was conducted using Scikit-learn's GridSearchCV. This method performs an **exhaustive search** over a specified parameter grid, evaluating every combination using cross-validation to identify the best configuration based on the selected evaluation metric in this case, the **R² score**.

K-Nearest Neighbors has several key parameters that significantly influence its performance:

- **n_neighbors**: Controls how many neighbors contribute to the prediction. A small **k** may lead to overfitting, while a large **k** may cause under fitting.
- **weights**: Determines how the influence of each neighbor is weighted. 'uniform' gives equal weight, while 'distance' gives more importance to closer neighbors.
- **p**: Defines the power parameter for the **Minkowski distance** metric:
 - **p = 1**: Manhattan distance
 - **p = 2**: Euclidean distance

By leveraging GridSearchCV, the model tuning process was made rigorous, data-driven, and fully integrated into the pipeline ensuring the final model is both robust and well-optimized for the housing price prediction task.



```

Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
      'Latitude', 'Longitude'],
      dtype='object'))]],
      ('knn', KNeighborsRegressor()))

Apply GridSearchCV to Tune Hyperparameters

#Define hyperparameter grid
param_grid = {
    'knn_n_neighbors': [3, 5, 7, 9],
    'knn_weights': ['uniform', 'distance'],
    'knn_p': [1, 2]
}
print(param_grid)

{'knn_n_neighbors': [3, 5, 7, 9], 'knn_weights': ['uniform', 'distance'], 'knn_p': [1, 2]}

Defining the evaluation metrics: R SQUARED

Automatic saving failed. This file was updated remotely or in another tab. Show diff
  
```

Fig 2.5.1:Output of param_grid

2.6: Use 5-Fold Cross-Validation

To ensure the robustness and generalizability of the model, **5-fold cross-validation** was used during the hyperparameter tuning process. Cross-validation is a reliable technique for assessing how a model will perform on unseen data by reducing the risk of overfitting and providing a more accurate estimate of model performance.

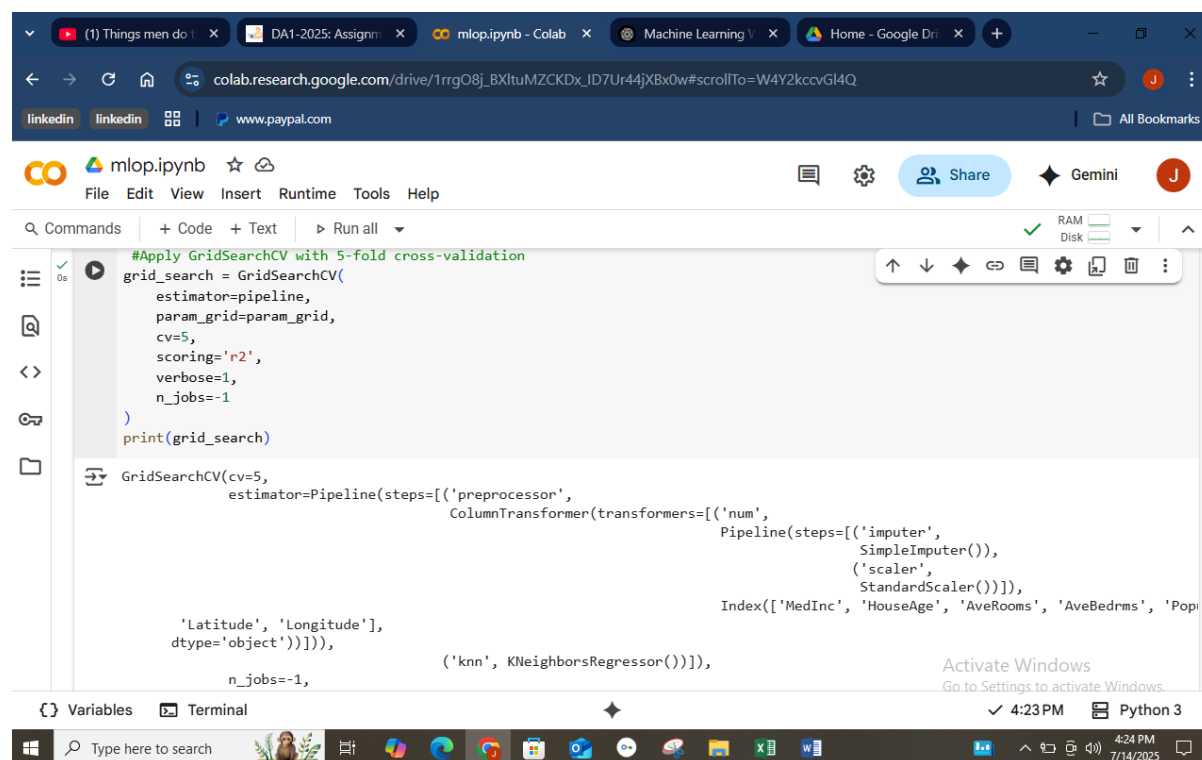
In 5-fold cross-validation, the training dataset is split into **five equal parts (folds)**:

- The model is trained on four folds and validated on the fifth.
- This process is repeated five times, with each fold serving as the validation set exactly once.
- The performance scores (in this case, **R² scores**) from each iteration are then **averaged** to produce a final, cross-validated score.

Why Use Cross-Validation?

- **Improved Evaluation:** It provides a more reliable estimate of model performance compared to a single train-test split.
- **Hyper parameter Tuning:** It is particularly useful during grid search to evaluate each hyper parameter combination multiple times.
- **Reduces Bias and Variance:** Balances the trade-off between training on more data and testing on more data.

By using 5-fold cross-validation, this project followed a best-practice approach for model selection and tuning, ensuring that the chosen model not only performs well on the training data but also generalizes effectively to new housing data scenarios.



```
#Apply GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='r2',
    verbose=1,
    n_jobs=-1
)
print(grid_search)

GridSearchCV(cv=5,
              estimator=Pipeline(steps=[('preprocessor',
                                         ColumnTransformer(transformers=[('num',
                                                                           Pipeline(steps=[('imputer',
                                                                           SimpleImputer()),
                                                                           ('scaler',
                                                                           StandardScaler()))],
                                                                           Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Popu
                                                                           'Latitude', 'Longitude'],
                                                                           dtype='object')))])),
              ('knn', KNeighborsRegressor()))],
              n_jobs=-1,
```

Fig 2.6.1: Output of grid_search

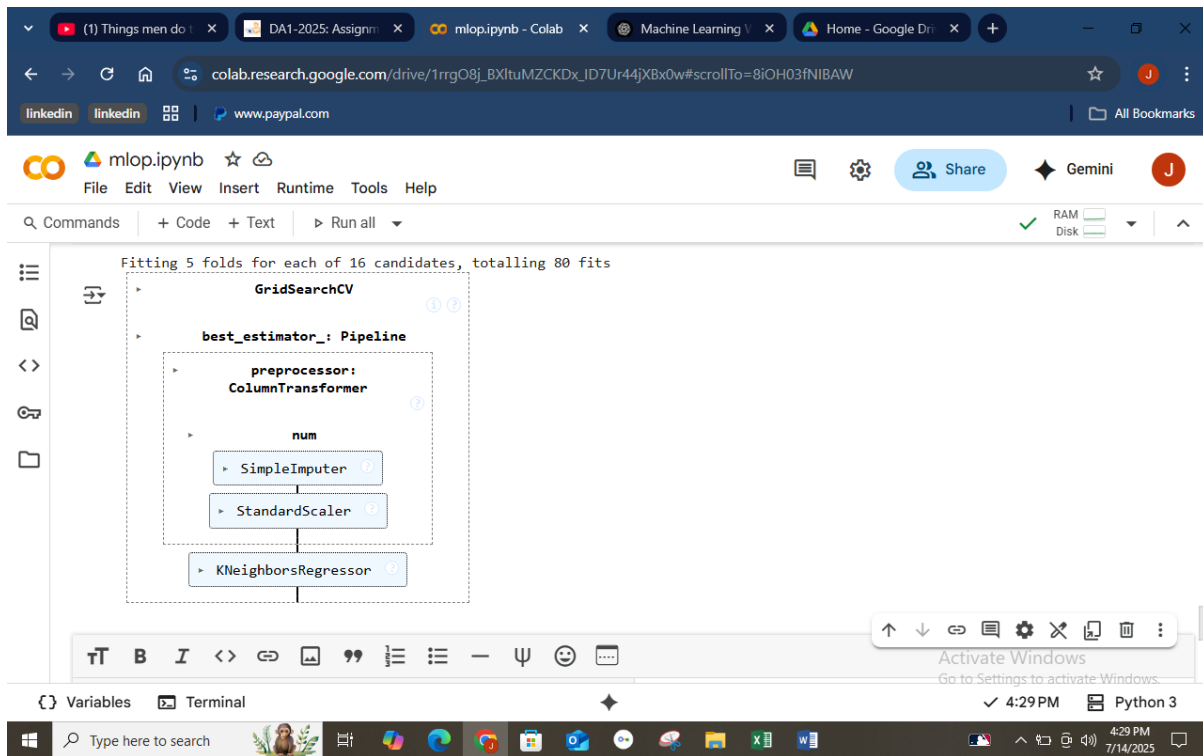


Fig 2.6.7: Output of fitting the model

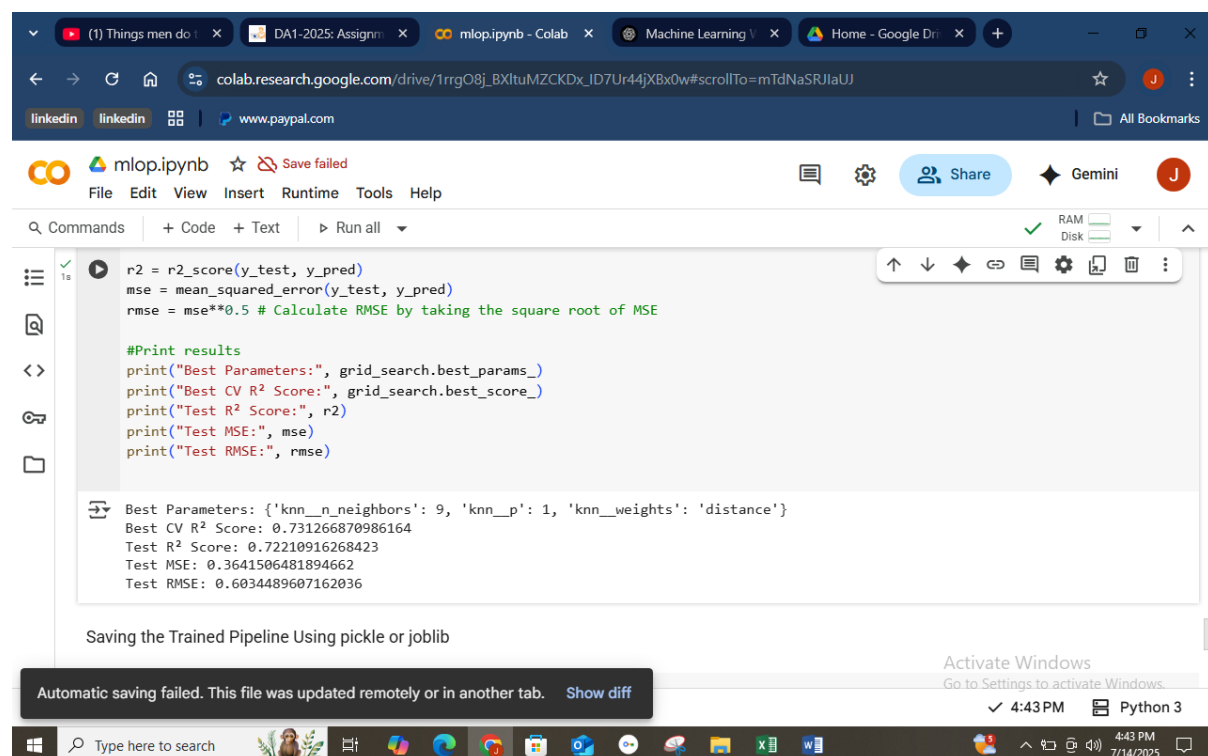
2.7: Define an Evaluation Metric: R² Score

To assess the performance of the regression model, the **R² Score (Coefficient of Determination)** was selected as the primary evaluation metric. This metric measures the proportion of the variance in the target variable that is predictable from the input features. It is widely used in regression tasks due to its interpretability and ability to indicate how well a model fits the data.

Why Use R² Score?

- **Interpretability:** R² provides a clear indication of how much of the variability in housing prices can be explained by the model.
- **Scale-independent:** Unlike error-based metrics, R² is not affected by the units of the target variable, making it easier to compare across models or datasets.
- **Performance indicator:**
 - An R² score of **1.0** indicates a perfect fit (model explains 100% of the variance).
 - A score of **0** means the model performs no better than the mean of the target values.
 - A **negative R²** suggests the model performs worse than a simple average prediction.

By using the R² score as the central metric, I ensured that model selection was guided by its ability to explain the variance in housing prices a critical consideration for decision-making in both business and academic contexts.



```

r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse**0.5 # Calculate RMSE by taking the square root of MSE

#Print results
print("Best Parameters:", grid_search.best_params_)
print("Best CV R2 Score:", grid_search.best_score_)
print("Test R2 Score:", r2)
print("Test MSE:", mse)
print("Test RMSE:", rmse)

```

```

Best Parameters: {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'distance'}
Best CV R2 Score: 0.731266870986164
Test R2 Score: 0.72210916268423
Test MSE: 0.3641506481894662
Test RMSE: 0.6034489607162036

```

Saving the Trained Pipeline Using pickle or joblib

Automatic saving failed. This file was updated remotely or in another tab. [Show diff](#)

Activate Windows
Go to Settings to activate Windows.

4:43 PM Python 3
7/14/2025

Fig 2.7.1: Output of the results

2.8: Save the Trained Pipeline Using `pickle` or `joblib`

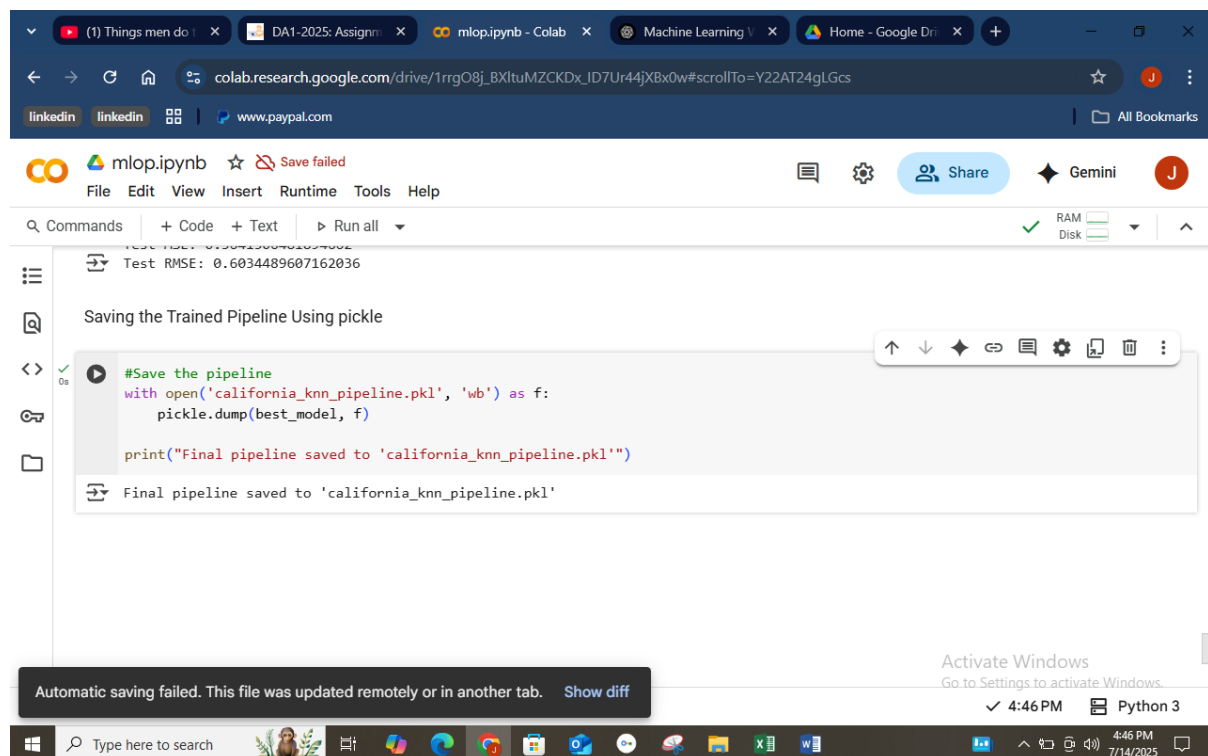
After training and tuning the final machine learning pipeline, the model was **saved to disk** using Python's `pickle` module. This step is critical for ensuring that the trained model can be reused in the future whether for making predictions, deploying to an application, or sharing with other teams **without the need to retrain it**.

Why Save the Model?

- **Reusability:** Enables fast loading of the trained model for real-time inference or further analysis.
- **Deployment-readiness:** Essential for integrating the model into production environments (e.g., web applications, APIs).
- **Consistency:** Ensures that the exact model and preprocessing steps used during training are preserved, reducing the risk of discrepancies during deployment.

Although `pickle` can also be used, `joblib` is preferred for models that contain large NumPy arrays (like in Scikit-learn pipelines) because it:

- Offers faster serialization and deserialization.
- Reduces file size for large objects.



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the filename 'mlop.ipynb', and a 'Save failed' notification. The notebook has tabs for 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The 'Runtime' tab is active, showing a 'Run all' button and a 'Commands' search bar. The notebook content shows a code cell with the following code:

```
#Save the pipeline
with open('california_knn_pipeline.pkl', 'wb') as f:
    pickle.dump(best_model, f)

print("Final pipeline saved to 'california_knn_pipeline.pkl'")
```

The output of the code cell shows the test RMSE: 0.6034489607162036 and the message: Final pipeline saved to 'california_knn_pipeline.pkl'. A notification at the bottom of the notebook says 'Automatic saving failed. This file was updated remotely or in another tab. Show diff'. The Windows taskbar is visible at the bottom, showing the time as 4:46 PM on 7/14/2025.

Fig 2.8.1: Output of saving the model

Link to Code:

https://colab.research.google.com/drive/1rrgO8j_BXltuMZCKDx_ID7Ur44jXBx0w?usp=sharing

Conclusion

This project successfully demonstrated the design and implementation of an end-to-end machine learning workflow to predict housing prices using the California Housing dataset. Through a structured approach that encompassed data preprocessing, model training, hyperparameter tuning, and evaluation, the project highlighted how statistical learning techniques can be applied to solve real-world regression problems.

The use of a **K-Nearest Neighbors Regressor** allowed us to model complex relationships between housing features and price, while techniques such as **GridSearchCV** and **cross-validation** ensured that the model was both optimized and robust. Preprocessing using **ColumnTransformer** and modular pipeline design facilitated a clean and reproducible workflow, emphasizing best practices in machine learning development.

Moreover, this assignment underscored the importance of **model interpretability**, **data quality** and **evaluation metrics** in building trustworthy AI systems. The project serves not only as a technical exercise but also as a practical example of how data science can deliver actionable insights and enhance decision-making in industries like real estate, finance, and public planning.

In conclusion, this project bridges the gap between theory and practice by translating classroom concepts into a deployable, business-relevant machine learning solution. It lays the groundwork for future enhancements such as feature importance analysis, model comparison and integration with larger data ecosystems, all of which contribute to more intelligent, data-informed strategies.